

Data structures

BINARY SEARCH TREES

Today's topics

- Binary trees
- Tree traversals
- Binary search trees

Motivation

What is the worst-case run time for element insertion into a sorted array?

$O(n)$

OK, but we can do better with a linked list...

But... what is the worst-case run time for element access in a linked list?

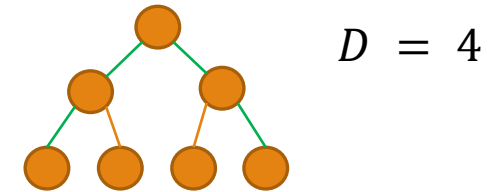
$O(n)$

Solution: non-linear data structure!

Question 1

Definition:

In a binary tree T , we define the diameter of the tree as the longest distance between any pair of nodes in T .



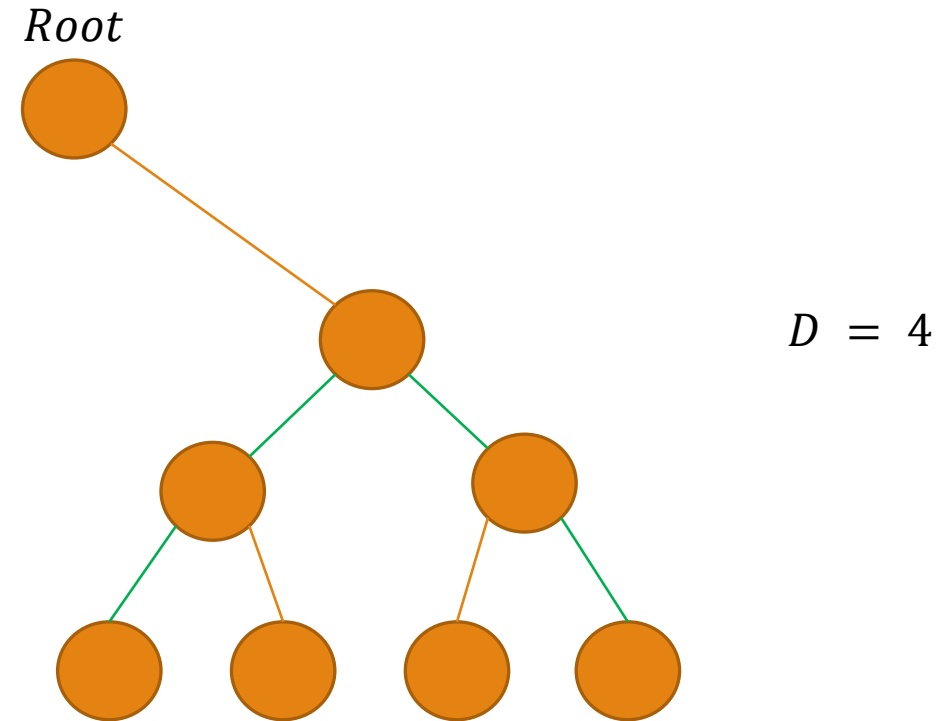
Prove or disprove:

The diameter of a binary tree T (with at least two nodes) is D ; then there exists a path of length D that goes through the root.

Question 1 - Solution

The claim is incorrect.

Consider the following tree:

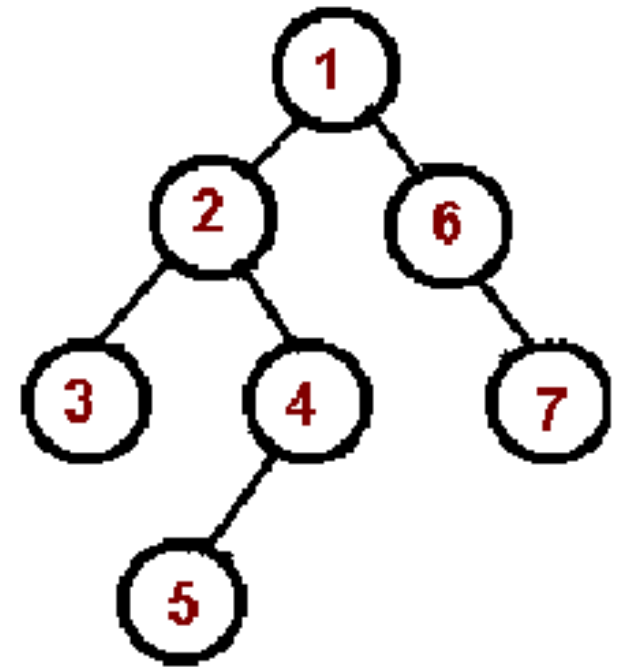


Binary trees traversals

Preorder:

- **Scan the root**
- Activate *preorder* on the left sub-tree (recursively)
- Activate *preorder* on the right sub-tree (recursively)

The root is
printed first

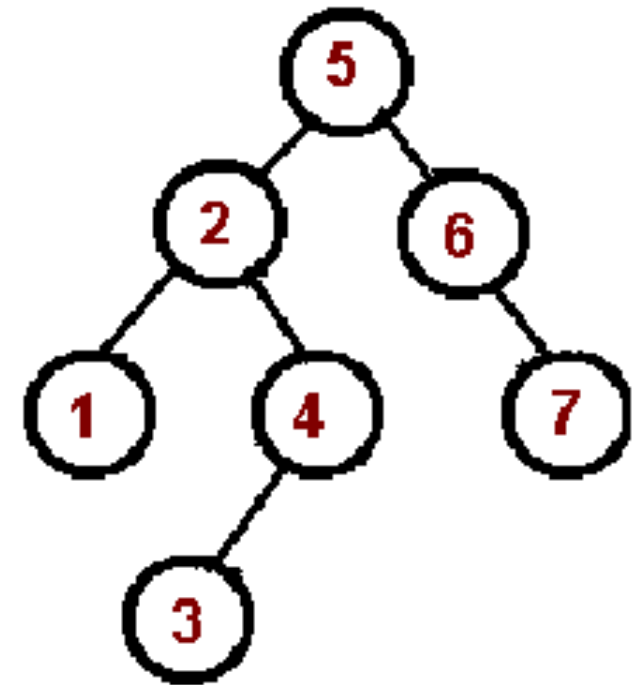


Binary trees traversals

Inorder

- Activate *inorder* on the left sub-tree (recursively)
- **Scan the root**
- Activate *inorder* on the right sub-tree (recursively)

The root
partitions
the scan

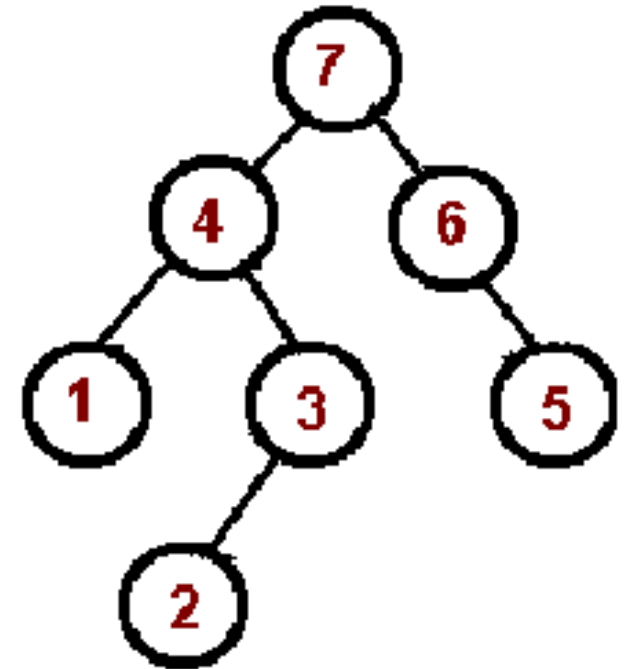


Binary trees traversals

Postorder:

- Activate *postorder* on the left sub-tree (recursively)
- Activate *postorder* on the right sub-tree (recursively)
- **Scan the root**

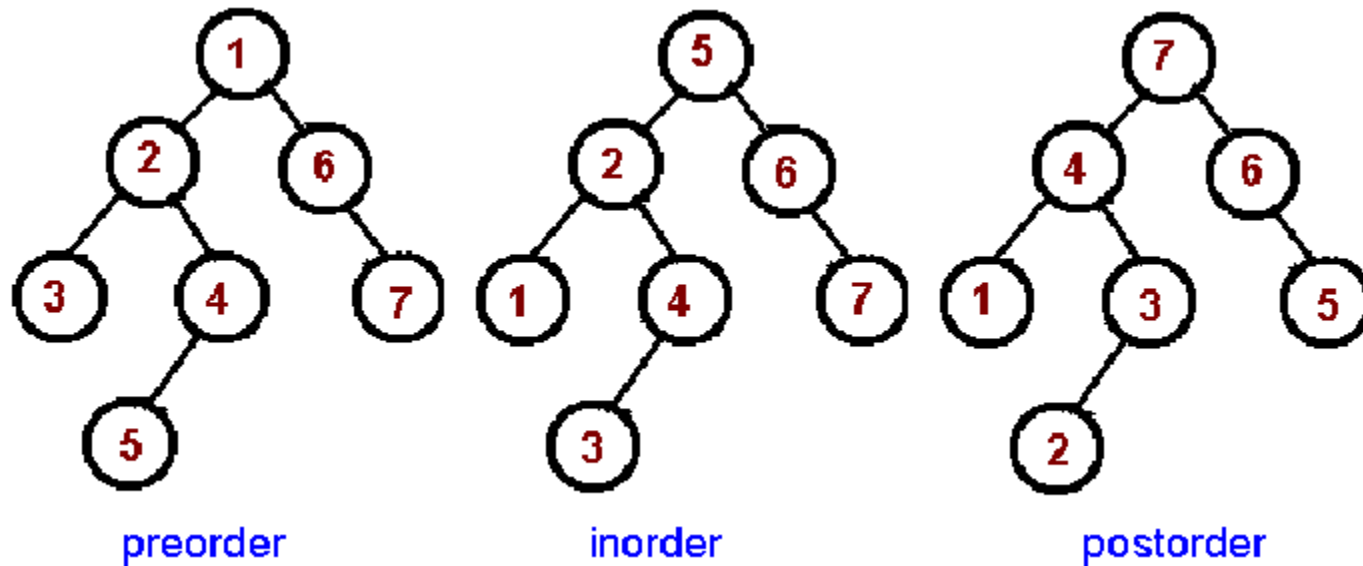
The root is
printed last



Binary Tree Traversals

What is the time complexity of a full traversal?

$O(n)$ – Because each element is processed once



Question 2

Given is the output of *inorder* and preorder traversals of the same binary tree T . Recreate T .

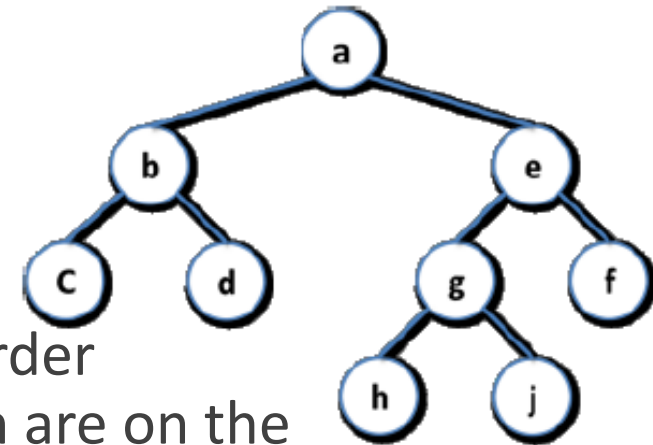
- Inorder: c, b, d, a, h, g, j, e, f
- Preorder: a, b, c, d, e, g, h, j, f

Question 2 - Solution

- Inorder: c, b, d, a, h, g, j, e, f
- Preorder: a, b, c, d, e, g, h, j, f

Solution

The first node in the preorder traversal is T 's root. Looking at the inorder traversal, we can see which nodes are on the left sub-tree, and which are on the right sub-tree (in inorder traversal, the root comes “between” the left subtree and the right subtree).



Question 3

Prove or disprove:

A binary tree can always be recreated uniquely based on the output of any single *traversal (preorder, inorder, or postorder)*.

Meaning – Given the pre/in/post-order traversal “a, b, c”, can you draw the shape of the original tree?

Question 3 - Solution

False.

Consider the following *preorder* traversal:

- Preorder: a, b, c

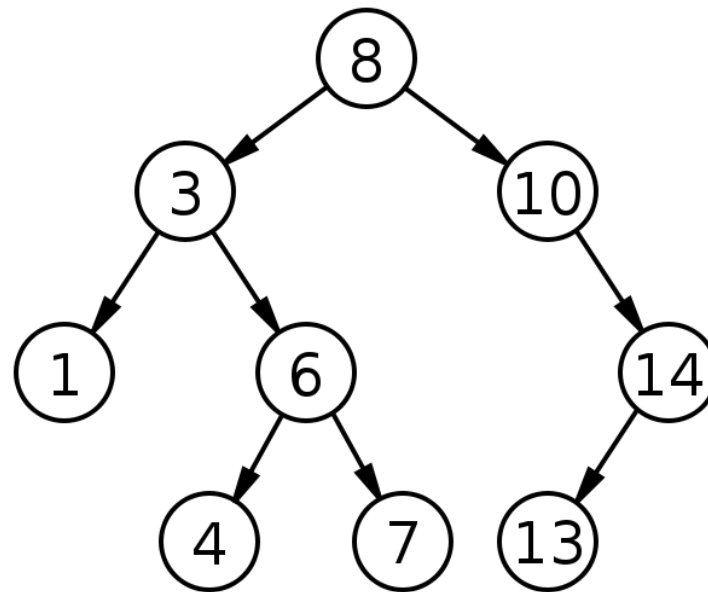


- You can come up with similar counterexamples for the other traversals.

Binary Search Tree

A binary tree where the following requirements hold:

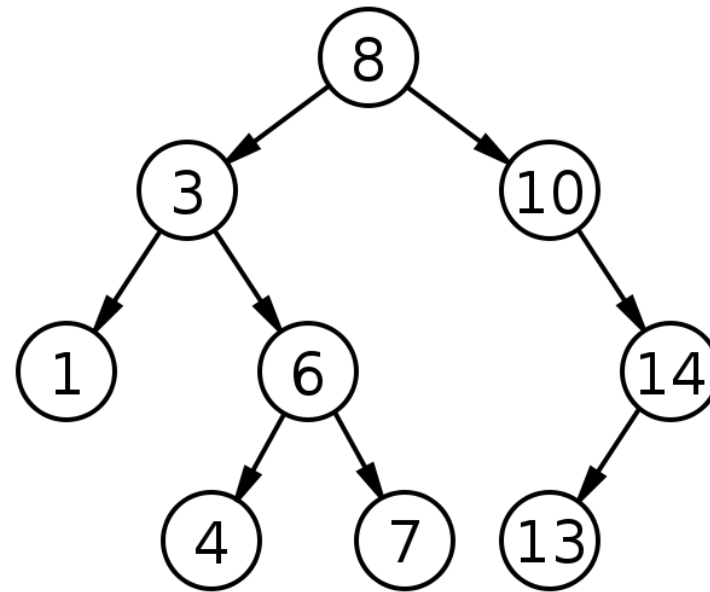
- The **value** of a node is **bigger** than all the values in its **left** subtree
- The **value** of a node is **smaller** than all the values in its **right** subtree



Binary Search Tree

- Insert 9 and 2 into the tree.
- Remove 8 from the tree.

Time complexity?



Question 4

You are searching for the number 363 in a BST.

Which of the following sequences cannot be the sequence of nodes examined?

- A. 2, 252, 401, 398, 330, 344, 397, 363
- B. 924, 220, 911, 244, 898, 258, 362, 363
- C. 925, 202, 911, 240, 912, 245, 363

Question 4 - Solution

You are searching for the number 363 in a BST.

Which of the following sequences cannot be the sequence of nodes examined ?

- A. 2, 252, 401, 398, 330, 344, 397, 363
- B. 924, 220, 911, 244, 898, 258, 362, 363
- C. 925, 202, 911, 240, 912, 245, 363

Question 5

Design an efficient algorithm to solve the following problem:

Given two numbers $a < b$, print in sorted order, all the keys in a BST that are in (a, b) .

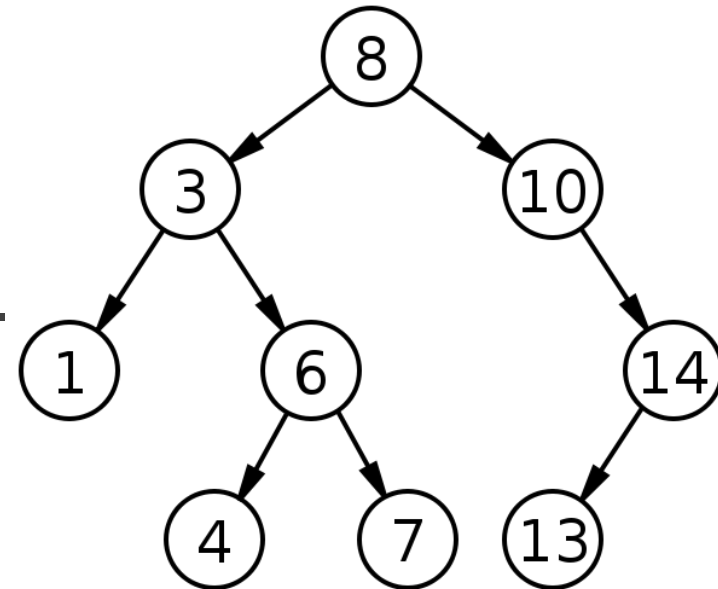
Question 5 - Solution

Design an efficient algorithm to solve the following problem:

Given two numbers $a < b$, print in sorted order, all the keys in a BST that are in (a, b) .

Solution:

Do an *inorder* traversal, but only visit subtrees that might contain values within the range (a, b) .



Question 5 - Solution

The idea is to adapt the in-order algorithm to this case:

```
find_range(a, b, root):
```

```
    If root == null:
```

```
        return
```

```
    If root.value > a:
```

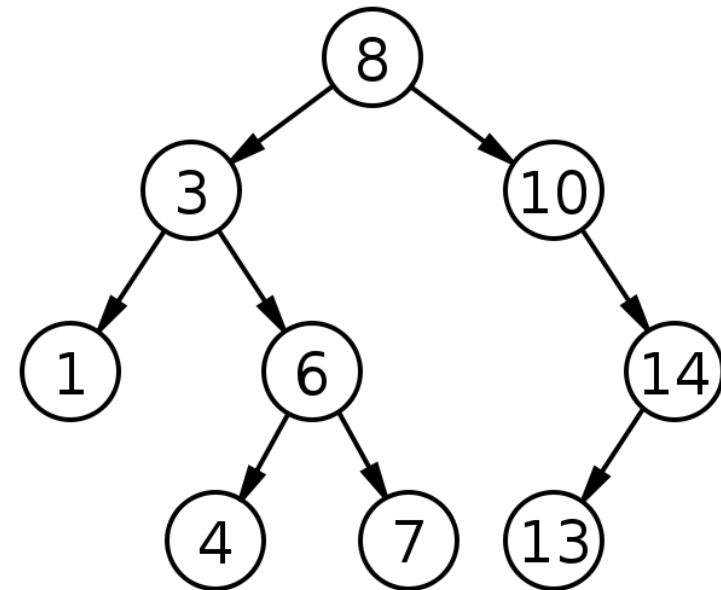
```
        find_range(a, b, root.left)
```

```
    If root.value > a && root.value < b:
```

```
        print root.value
```

```
    If root.value < b:
```

```
        find_range(a, b, root.right)
```



Question 6

Prove that in a binary search tree an *inorder* traversal outputs a sorted array.

Question 6 - Solution

We will prove the above by induction on the number of nodes in the tree.

Base: The claim clearly holds for an empty tree and for a tree with only 1 node.

Hypothesis: Assume that the claim is true for any $k < n$ nodes.

Induction step:

Prove for a tree with n nodes. Let r be the key of the root. Since the tree is a BST all keys smaller than r are in the left sub-tree, and all keys bigger than r are in the right sub-tree. By the inductive hypothesis, applying *inorder* on the left sub-tree outputs a sorted array of the keys of the left sub tree (why?), and the same for the right sub tree. Then, by definition of inorder, the output will consist of the concatenation of the output for the left sub-tree, then r , then the output for the right sub-tree, so we get a sorted array with all n keys.

Question 7 – maximal path

T is a binary tree with n vertices. Each vertex x has the following fields:

x.key – a natural number

x.left – pointer to the left child

x.right – pointer to the right child

Define:

$\text{maxSumPath}(x)$ – the maximal sum of keys in a (simple) path from the root to some leaf.

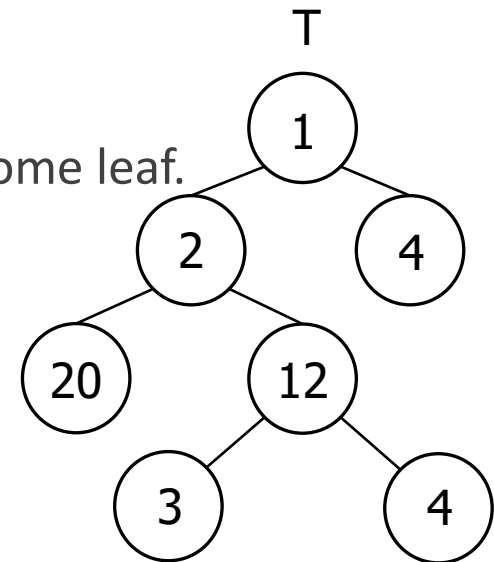
$\text{MaxPath}(x)$ – some path with this sum.

Example:

$\text{maxSumPath}(x) = 23$

$\text{MaxPath}(x) = \text{left, left}$

x is the root of T



Question 7 – maximal path

Describe an algorithm for finding the value $maxSumPath(x)$ in $\theta(n)$ time when x is the root of T .

Solution:

Recursively compute the $maxSumPath$ of the left child and also the $maxSumPath$ of the right child, and add the key of the vertex to the maximum among them.

```
maxSumPath(x)
  if x = NULL
    return 0
  Lmax <- maxSumPath(x.left) // left-sub-tree(T)
  Rmax <- maxSumPath(x.right) // right-sub-tree(T)
  m <- Max(Lmax, Rmax)
  return m+x.key
```


Question 7 – maximal path

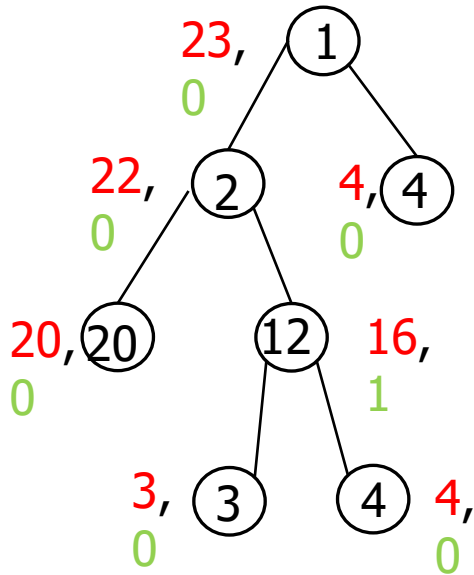
We want to write a function that prints a path as described in the previous section, after a call to the function *maxSumPath*. How can this be done?

Hint: We can modify the algorithm from section 1 and add memory

Solution:

We will add a field in the vertex structure, **x.val**. We will calculate *maxSumPath* as before, only we will update **x.val** to 0 if the maximal sum comes from the left sub-tree and 1 if from the right. In printing the path we will use the value **x.val** to decide which direction to continue.

Question 7 – maximal path



maxSumPath(x)

```
if x = NULL
    return 0
Lmax ← maxSumPath(x.left) // left-sub-tree(T)
Rmax ← maxSumPath(x.right) // right-sub-tree(T)
m ← Max(Lmax, Rmax)
if m = Lmax
    x.val ← 0
else
    x.val ← 1
return m+x.key
```

printPath(T)

```
maxSumPath(T)
y ← T
while (y.left is not NULL) or (y.right is not NULL)
    if y.val = 0
        print "left"
        y ← y.left
    else
        print "right"
        y ← y.right
```

Question 7 – maximal path

1. Describe an algorithm for finding the value $maxSumPath(x)$ when x is the root of the tree T in $\theta(n)$ time.
2. How can the algorithm be modified so that it also prints the maximal path?

Time complexity:

1. We visit each vertex at most twice, once in the recursive call (going down the tree) and an additional time upon return, therefore the running time is $\theta(n)$.
2. We visit each vertex in the path once, and the number of vertices in the path is bounded by h – the height of the tree. Therefore a total of $\theta(n)$.

Note: the height of the tree, h , belongs to $O(n)$.

Conclusions from question 7

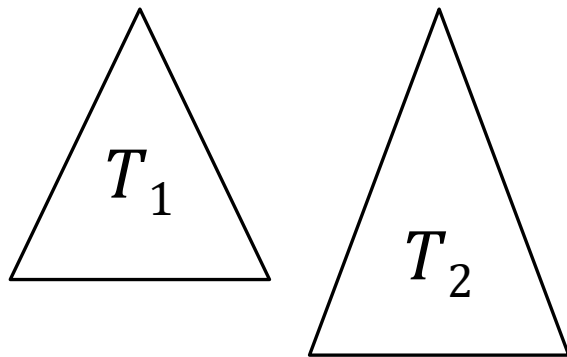
Adding a field that "remembers" events that occurred in the structure is a recurring technique. In particular, this is a very important tool for reconstructing solutions while calculating the value of solutions in optimization problems.

Question 8

Given two binary search trees T_1 and T_2 with heights h_1 and h_2 respectively (h_1 and h_2 are given).

It is additionally given that all values in T_1 are strictly smaller than all values in T_2 . Assume all key values are distinct.

1. How can the two trees be merged into a single binary search tree containing the union of the values in time $\theta(\underline{\min}(h_1, h_2))$?
2. What will be the height of the merged tree?



Question 8

How can the two trees be merged into a single binary search tree containing the union of the values in time $\theta(\min(h_1, h_2))$?

Solution:

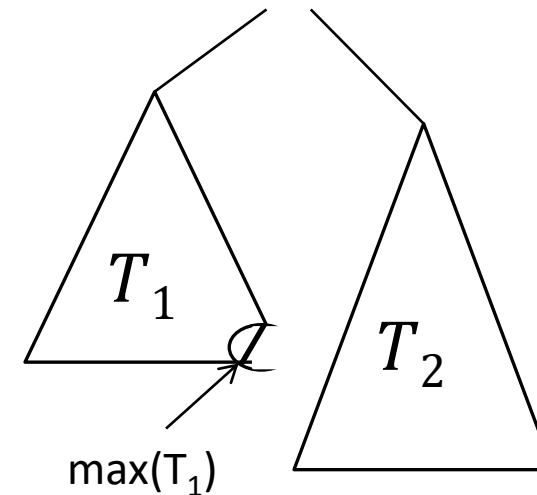
case a: $h_1 \leq h_2$

Extract from T_1 the element v with the maximum key in T_1 in $O(h_1)$ time.

$v.\text{left} \leftarrow T_1$

$v.\text{right} \leftarrow T_2$

v is the root of the new merged tree.



Question 8

How can the two trees be merged into a single binary search tree containing the union of the values in time $\theta(\min(h_1, h_2))$?

Solution:

case a: $h_1 \leq h_2$

Extract from T_1 the element v with the maximum key in T_1 in $O(h_1)$ time.

$v.\text{left} \leftarrow T_1$

$v.\text{right} \leftarrow T_2$

v is the root of the new merged tree.

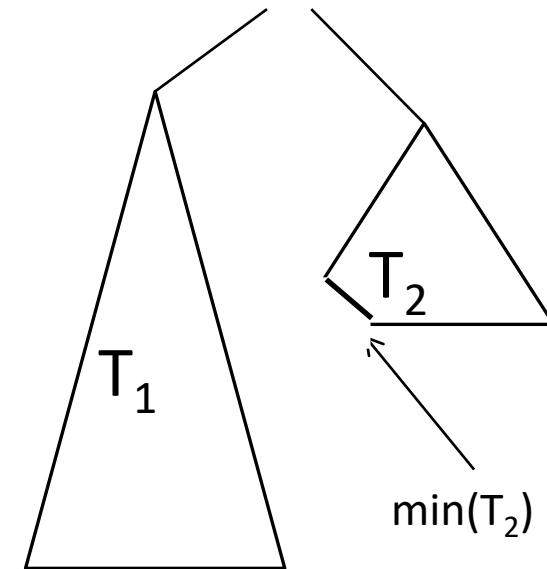
case b: $h_1 > h_2$

Extract from T_2 the element v with the minimum key in T_2 in $O(h_2)$ time.

$v.\text{left} \leftarrow T_1$

$v.\text{right} \leftarrow T_2$

v is the root of the new merged tree.



Question 8

What will be the height of the merged tree?

Solution:

$$\max(h_1, h_2) + 1$$

Question 8

What would we do if the heights of the trees were not given to us?

Question 8

What would we do if the heights of the trees were not given to us?

Solution Intuition: Interleaved Traversal

The Challenge: Since the heights are unknown, fully traversing one tree first risks taking $\theta(\underline{\mathbf{max}}(h_1, h_2))$ time if we accidentally choose the taller tree.

The Strategy: We use a simultaneous, step-by-step traversal. We advance a pointer in T_1 (looking for the max) and a pointer in T_2 (looking for the min), alternating exactly one step at a time.

The Stop Condition: We halt the entire process the moment *either* pointer reaches a leaf.

The Complexity: Because we stop as soon as the shorter path ends, we take at most $\underline{\mathbf{min}}(h_1, h_2)$ steps in each tree. This guarantees the required $\theta(\underline{\mathbf{min}}(h_1, h_2))$ time complexity without needing to know the heights in advance.

Conclusions from question 8

We saw how two trees can be easily joined as children of a common parent.

We used the given information about the relations between the key sizes to maintain the search tree property during the join.

Question 9 – If time permits

Find an algorithm that checks if a given binary tree T is a BST.

Assume that all values are distinct integers.

Assume that each vertex has a pointer to its two children but no pointer to its parent.

Question 9

Find an algorithm that checks if a given binary tree T is a BST.

Assume that all values are distinct integers.

Assume that each vertex has a pointer to its two children but no pointer to its parent.

Solution (wrong):

We will recursively check at each vertex that the value of the left child is smaller than the current value and the value of the right child is larger than the current value.

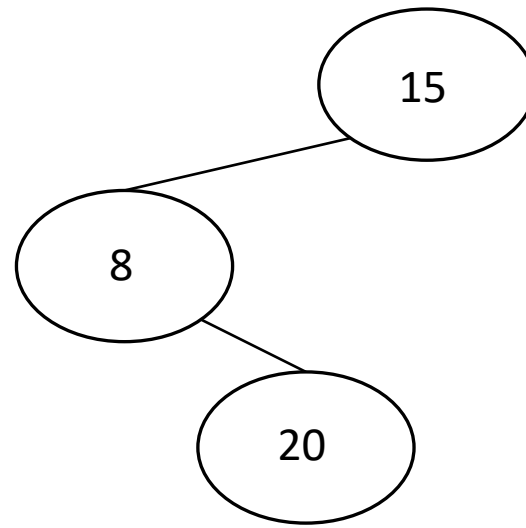
This solution is not sufficient – find a counterexample

Question 9

Solution (wrong):

We will recursively check at each vertex that the value of the left child is smaller than the current value and the value of the right child is larger than the current value.

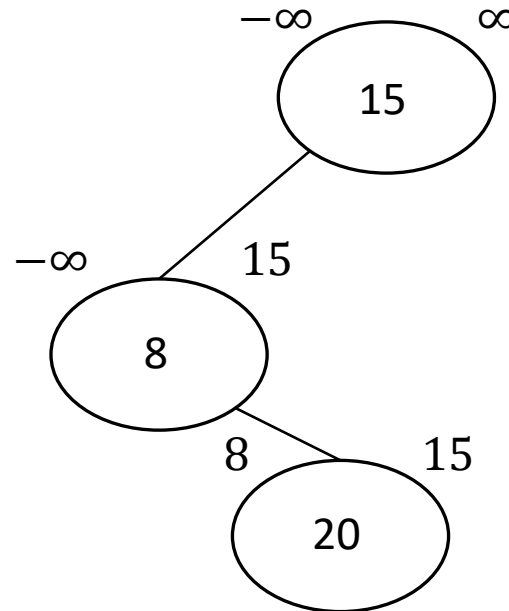
This solution is not sufficient – find a counterexample



Question 9

Solution (corrected):

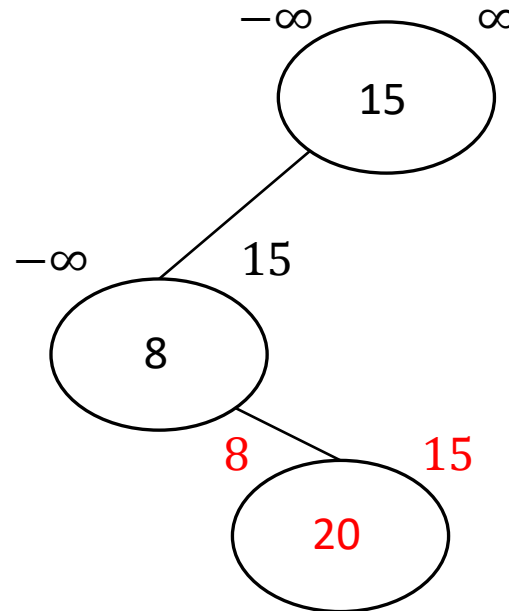
We will use a helper function that maintains the upper and lower bounds of the allowed values in the sub-tree



Question 9

Solution (corrected):

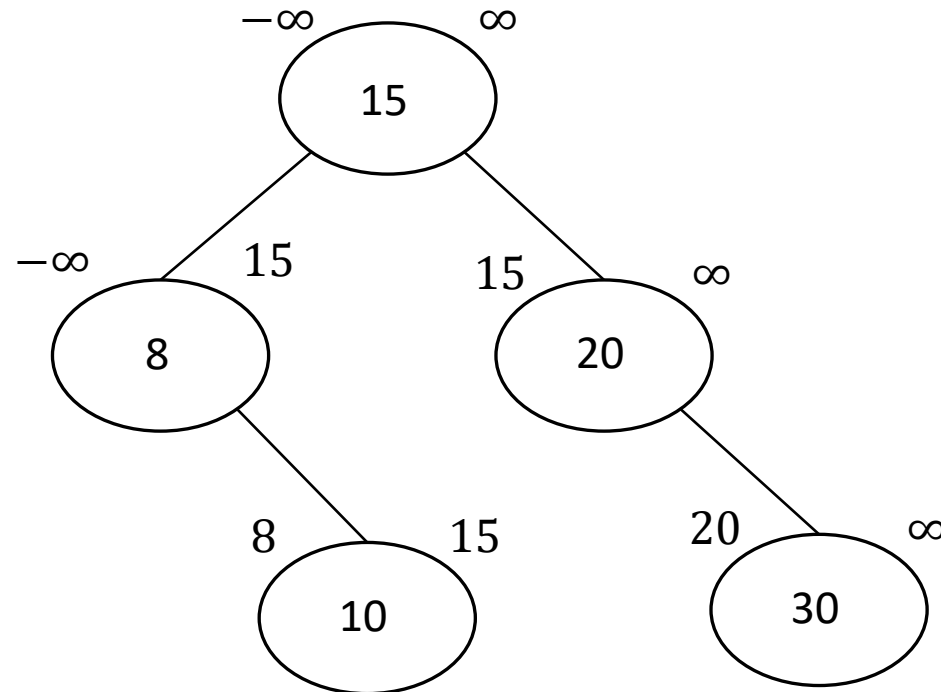
We will use a helper function that maintains the upper and lower bounds of the allowed values in the sub-tree



Question 9

Solution (corrected):

We will use a helper function that maintains the upper and lower bounds of the allowed values in the sub-tree



Question 9

Solution (corrected):

We will use a helper function that maintains the upper and lower bounds of the allowed values in the sub-tree

isValid(T)

```
return isValidHelper(root, -infinity, +infinity)
```

isValidHelper(current, min, max)

```
if curr.value < min or curr.value > max  
    return FALSE
```

```
if curr.left ≠ NULL and not isValidHelper(curr.left, min, curr.value)  
    return FALSE
```

```
if curr.right ≠ NULL and not isValidHelper(curr.right, curr.value, max)  
    return FALSE
```

```
return TRUE
```

Question 9

isValid(T)

```
return isValidHelper(root, -infinity, +infinity)
```

isValidHelper(current, min, max)

```
if curr.value < min or curr.value > max
```

```
    return FALSE
```

```
if curr.left ≠ NULL and not isValidHelper(curr.left, min, curr.value)
```

```
    return FALSE
```

```
if curr.right ≠ NULL and not isValidHelper(curr.right, curr.value, max)
```

```
    return FALSE
```

```
return TRUE
```

Time Analysis:

Runtime is $\theta(n)$.

Another possible solution is performing an inorder traversal and checking if the values are increasing.